

Simulación

LLMs en Agent-Based Models

Estructura de la sesión

180 minutos · 8 secciones

Sección	Duración	Formato
1. Introducción y hook	15 min	Exposición + video
2. Fundamentos de ABM tradicionales	25 min	Teoría + demo
3. LLMs: arquitectura y capacidades	20 min	Teoría + ejemplos
4. Integración de LLMs en ABM	35 min	Teoría + demo en vivo
5. Aplicaciones y casos de estudio	30 min	Ejemplos reales + videos
6. Desafíos, limitaciones y evaluación	20 min	Discusión
7. Actividad práctica guiada	35 min	Trabajo en grupo / individual
8. Futuro, tendencias y conclusión	15 min	Cierre + Q&A

1. Introducción

¿Y si los agentes pudieran *razonar, recordar y conversar*?



INGENIERÍA

¿Qué pasaría si los agentes de una simulación pudieran **razonar, recordar y conversar** como humanos en lugar de seguir reglas rígidas?

Una breve historia

De reglas rígidas a mentes simuladas

- **1971** — Schelling publica el **Modelo de Segregación**: agentes simples, reglas locales, comportamiento emergente.
- **1990s-2000s** — **Sugarscape**, **NetLogo**, **Mesa**: ABM se vuelve accesible.
- **2017** — Vaswani et al.: arquitectura **Transformer** (**Attention Is All You Need**).
- **2022-2023** — ChatGPT, GPT-4, Llama. Los LLMs superan benchmarks de razonamiento.
- **2023** — Park et al., **Generative Agents** (Stanford "Smallville"): 25 agentes LLM viven, recuerdan y se relacionan.
- **2024-2026** — Frameworks LLM-ABM, simulaciones a escala, agentes multimodales.

¿Por qué importa hoy?

Dos curvas que se cruzan

- **Explosión de los LLMs**
 - Razonamiento, memoria y **tool use** a costo cada vez menor
 - Modelos open-source ejecutables localmente
- **Necesidad de simulaciones más realistas**
 - Modelos económicos / sanitarios con agentes que **se equivocan, dudan y aprenden**
 - Políticas públicas evaluadas sobre sociedades sintéticas
 - Hipótesis testeables en mundos **bottom-up**

*El cuello de botella ya no es la teoría, es la **fidelidad cognitiva** de los agentes.*

Objetivos de la sesión

Al final de la clase vas a poder...

- **Identificar** los componentes clásicos de un ABM y sus limitaciones
- **Reconocer** qué capacidades de los LLMs los hacen útiles como agentes
- **Diferenciar** los 4 niveles de integración LLM + ABM
- **Diseñar** un agente generativo simple con **persona**, memoria y prompt
- **Evaluar** críticamente costos, sesgos y validez de un LLM-ABM
- **Implementar** una mini-simulación durante la actividad práctica

Antes de empezar...

Una encuesta rápida

- ¿Cuántos **ya usaron ABM** (NetLogo, Mesa, Repast)?
- ¿Cuántos **integraron LLMs** vía API en algún proyecto?
- ¿Cuántos **leyeron Generative Agents** de Park et al. (2023)?
- ¿Cuántos están acá por **curiosidad**, **researcher**, **hacking** o un **proyecto concreto**?

2. ABM tradicionales

Fundamentos del modelado *bottom-up*



INGENIERÍA

¿Qué es un Agent-Based Model?

Modelar el todo desde las partes

Un **ABM** es una técnica de simulación donde:

- Cada **agente** es una entidad **autónoma** con estado y reglas propias
- Los agentes **interactúan localmente** con su entorno y entre sí
- El **comportamiento global emerge** de esas interacciones — no se programa

*Filosofía **bottom-up**: ingeniamos los individuos, observamos la sociedad.*

Los cuatro ingredientes

Anatomía de un ABM clásico

- **Agentes** — heterogéneos, con estado interno y reglas de decisión
- **Entorno** — espacial (grilla, red, continuo) o abstracto
- **Reglas de interacción** — cómo perciben, deciden y actúan
- **Bucle de simulación** — pasos discretos de tiempo, scheduling

En código: típicamente una clase `Agent`, una clase `Model`, y un `step()` que avanza el reloj.

Schelling (1971)

Segregación a partir de tolerancia mínima

- Agentes en una grilla, dos colores
- Regla: "si menos del X % de mis vecinos es como yo, me mudo"
- Para tolerancias **moderadas** ($X \approx 30\%$)...
 - ...emerge **segregación masiva**

*Ningún agente quiere segregar. La segregación es **emergente**.
Una de las primeras pruebas de que las reglas locales generan estructuras globales sorprendentes.*

Sugarscape y SIR

Otros modelos de referencia

Sugarscape (Epstein & Axtell, 1996)

- Agentes recolectan **azúcar** en una grilla con depósitos
- Estudia desigualdad, comercio, herencia, cultura

SIR en agentes (epidemias)

- Estados: **Susceptible** → **Infected** → **Recovered**
- Cada agente decide moverse, contactar, contagiar
- Permite estudiar **intervenciones heterogéneas** que un SIR clásico no captura

Herramientas populares

El ecosistema de ABM

Herramienta	Lenguaje	Fortalezas
NetLogo	propio	Didáctico, GUI, miles de modelos
Mesa	Python	Integración con el stack ML/data
Repast	Java	Escalabilidad, HPC
AnyLogic	Java/UI	Industria, multi-paradigma
Agents.jl	Julia	Performance, sintaxis moderna

*Para LLM-ABM, **Mesa** es el camino natural: Python = mismo lenguaje que las APIs de LLM.*

Demo: **flocking** en Mesa

Boids — comportamiento emergente sin líder

```
class Boid(mesa.Agent):
    def step(self):
        neighbors = self.model.space.get_neighbors(
            self.pos, self.vision, include_center=False
        )
        if not neighbors:
            return
        cohesion = self._cohesion(neighbors)
        separation = self._separation(neighbors)
        alignment = self._alignment(neighbors)
        self.velocity += cohesion + separation + alignment
        self.velocity = self._normalize(self.velocity, max_speed=2.0)
        self.model.space.move_agent(self, self.pos + self.velocity)
```

Tres reglas locales → bandadas, vórtices, splits y merges.

Ventajas y limitaciones

ABM tradicional, sin LLMs

Ventajas	Limitaciones
Captura heterogeneidad y emergencia	Reglas rígidas , poco realistas
Bottom-up y mecanicista	Difícil modelar lenguaje o creatividad
Permite experimentos contrafactuales	Calibration costosa con datos reales
Escalable a millones de agentes	Agentes no aprenden dentro del modelo
Reproducibile, abierto a inspección	Espacio de behaviors limitado a priori

*Estas limitaciones son **exactamente** lo que los LLMs vienen a resolver.*

3. Large Language Models

Arquitectura y capacidades relevantes para ABM

Transformers

Breve repaso

- Un **Transformer** procesa secuencias de **tokens** mediante capas de **self-attention**
- Cada token "mira" a todos los anteriores y pondera su relevancia
- Pre-entrenamiento: predecir el siguiente token sobre **billones** de palabras
- **Fine-tuning** + **RLHF**: alinear con instrucciones humanas

*Para nosotros: una **caja que dado un prompt devuelve texto coherente y razonado**.
No nos importa **cómo** lo hace — sí qué **capacidades** exhibe.*

Capacidad 1 · Razonamiento

Chain-of-Thought y ReAct

- **Chain-of-Thought (CoT)** — "piensa paso a paso" antes de responder
- **ReAct** — alterna **Reason** y **Act**: el agente razona, ejecuta una herramienta, observa el resultado, sigue razonando

```
Pregunta: ¿Debería comprar la acción AAPL hoy?  
Pensamiento: Necesito el precio actual y los últimos earnings.  
Acción: get_stock_price("AAPL")  
Observación: 187.34  
Pensamiento: Comparo con la media móvil de 50 días...  
Acción: get_ma("AAPL", days=50)  
Observación: 182.10 → tendencia alcista.  
Respuesta: Sí, con stop-loss en 184.
```

Capacidad 2 · Memoria y reflexión

Más allá de la ventana de contexto

- **Memoria a corto plazo** — el contexto que se le pasa en el prompt
- **Memoria a largo plazo** — vectores en una base externa (RAG)
- **Reflexión** — el agente periódicamente **resume** sus experiencias y genera **insights** de alto nivel
 - "He sido rechazado tres veces por María → debería cambiar de estrategia"

*Park et al. (2023) muestran que **reflexión + recuperación por relevancia** es lo que hace creíbles a los **Generative Agents**.*

Capacidad 3 · Planificación multi-paso

Del objetivo a la acción concreta

- LLMs pueden **descomponer un objetivo** en sub-tareas
- **Tree of Thoughts**, **Plan-and-Solve**, **Self-Refine**
- Útil para agentes que persiguen **metas a largo plazo**:
 - Estudiar para un examen
 - Negociar un contrato
 - Optimizar una ruta de delivery

*El plan no es estático: el agente lo **revisa** ante observaciones nuevas.*

Capacidades 4 y 5 · Lenguaje y herramientas

Comunicarse e interactuar con el mundo

Lenguaje natural

- Los agentes pueden **conversar entre sí** y con el investigador
- Permite **prompting** del experimentador en pleno **runtime**

Tool use / Function calling

- El LLM emite llamadas estructuradas a funciones externas (APIs, calculadora, base de datos)
- Habilita **agentes multi-agente colaborativos** (ej. AutoGen, CrewAI)

Base model vs agentic LLM

No es lo mismo "un LLM" que "un agente LLM"

- Un **base model** completa texto. Punto.
- Un **agentic LLM** integra:
 - **Memoria** persistente
 - **Herramientas** (funciones, APIs, código)
 - **Bucle** percepción → razonamiento → acción
 - **Política** de cuándo parar, replanificar, escalar

*En LLM-ABM lo que necesitamos es la **versión agentic**, no el chat puro.*

Modelos recomendados (2026)

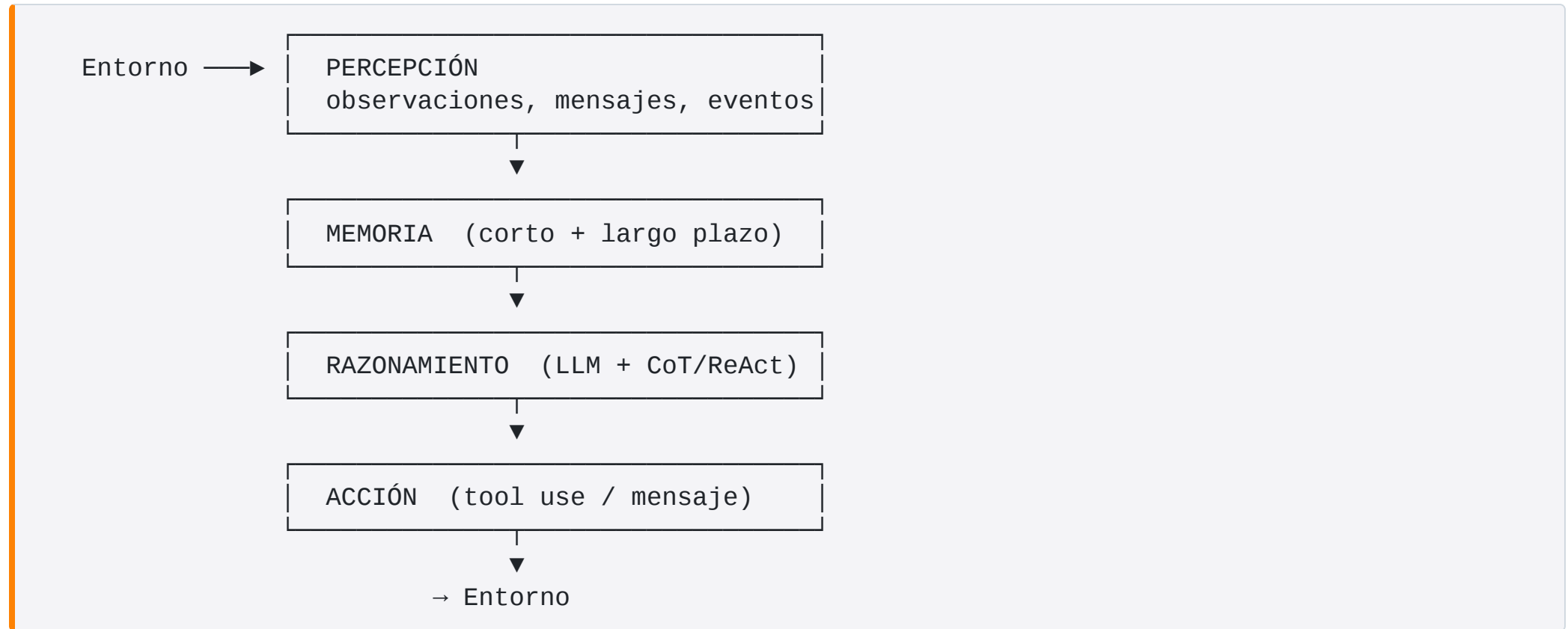
El paisaje hoy

Familia	Tamaño / acceso	Notas
Llama 4	open-weights	Razonable corriendo local con Ollama
Claude 4	API	Largos contextos, fuerte en reasoning
GPT-4o / 5	API	Multi-modal, function calling maduro
Grok 3	API	Bueno en exploración / long context
Mistral / Qwen	open-weights	Para agentes locales muy baratos
DeepSeek-R1	open-weights	Reasoning explícito, ideal CoT

Regla práctica: **prototipar con API, escalar con modelo local.**

Anatomía de un agente LLM

El bucle canónico



4. Integración LLM + ABM

Métodos y arquitecturas

Cuatro niveles de integración

¿Cuánto LLM querés?

1. **Asistente de desarrollo** — el LLM ayuda a escribir el ABM
2. **Capa de decisión** — query al LLM en cada paso del agente
3. **Agentes generativos completos** — memoria + reflexión + planning
4. **Híbridos** — reglas + LLM en los puntos donde aporta más

Más LLM → más realismo, más costo, menos reproducibilidad.

Nivel 1 · Asistente de desarrollo

El LLM no está dentro del modelo, está en tu IDE

- Generás clases, **step functions**, **visualizers** con ayuda del LLM
- Útil para **acelerar** el ABM tradicional, no para enriquecerlo
- Riesgos:
 - Código sutilmente incorrecto
 - Dependencias innecesarias
 - "**Stack Overflow** generativo"

Es lo más barato y, en general, lo primero que un equipo prueba.

Nivel 2 · LLM como capa de decisión

Una llamada por agente, por step

```
def step(self):  
    obs = self.percibir(self.model)  
    decision = llm.complete(  
        prompt=f"Eres {self.persona}. Estado: {obs}. ¿Qué haces?",  
        schema=ActionSchema,  
    )  
    self.ejecutar(decision)
```

- Más fácil de implementar que un agente generativo completo
- **Costoso**: 1 llamada × N agentes × T pasos
- Ideal para sub-conjuntos pequeños de agentes "humanos" rodeados de agentes simples

Nivel 3 · Agentes generativos completos

La arquitectura de Park et al. (2023)

- **Memory stream** — log de observaciones con timestamp e importancia
- **Retrieval** — $\text{recencia} \times \text{importancia} \times \text{relevancia (similitud)}$
- **Reflection** — síntesis periódica de **insights** de alto nivel
- **Planning** — agenda diaria que se reescribe ante eventos
- **Acción** — generación de comportamientos en lenguaje natural

*Costoso, lento, pero **el único nivel donde emerge** el comportamiento social rico que vimos en Smallville.*

Nivel 4 · Híbridos

Reglas + LLM, lo mejor de ambos mundos

- La **mayoría** del comportamiento es **regla** (rápido, barato, reproducible)
- El LLM se invoca **solo** cuando:
 - el agente enfrenta una situación **fuera de distribución**
 - hace falta **diálogo** o **negociación**
 - se necesita **interpretar** texto (noticias, posts, contratos)
- **Fallback** a reglas si el LLM falla o excede budget

*En producción, **casi todos** los LLM-ABM serios son híbridos.*

Frameworks y herramientas

El stack actual (2026)

- **Mesa-LLM** — **bindings** directos sobre Mesa, agentes con prompt + memoria
- **LangChain / LangGraph + Mesa** — orquestación de cadenas y **tools**
- **AutoGen** — multi-agente conversacional, **Microsoft**
- **CrewAI** — agentes con roles colaborativos
- **Shachi, AgentGPT** — frameworks específicos de agentes
- **Ollama / LM Studio** — modelos locales para escalar sin sangrar la billetera
- **NetLogo + Python bridge** — para reusar modelos clásicos

Plantilla de prompt

Un agente consumidor en un mercado

```
agent_prompt = ""
Eres un consumidor en un mercado simulado.

Personalidad: {persona}
Memoria reciente: {memoria}
Situación actual: {estado}
Productos disponibles: {productos}
Tu presupuesto: {presupuesto}

Decide qué hacer en este turno y por qué.
Responde EXCLUSIVAMENTE como JSON con la forma:

{{
  "accion": "comprar" | "esperar" | "negociar",
  "producto_id": int | null,
  "razon": str,
  "estado_emocional": str
}}
```

Demo · Lo que vamos a observar

Mercado con 15 agentes generativos

- **Personas** distintas: ahorrativo, impulsivo, escéptico, **fashion-victim**, etc.
- **Memoria** compartida: precios pasados, decepciones, recomendaciones
- **Métricas a mirar:**
 - Distribución de compras por **persona**
 - Aparición espontánea de **clusters** de gusto
 - **Influencia social** entre agentes que se comunican
 - **Volatilidad** de precios vs un baseline reglas-only

Output esperado: *dinámicas que un ABM tradicional sólo logra con muchos parámetros ajustados a mano.*

Optimización: cómo no fundirte la cuenta

Técnicas para escalar LLM-ABM

- **Archetypes / clustering** — agrupar agentes parecidos, una llamada por **cluster**
- **Distilación** — entrenar un modelo chico que imite al grande
- **Batching** — N prompts → 1 llamada
- **Caching** — situaciones repetidas no se vuelven a consultar
- **Modelos locales** — Llama / Mistral con Ollama, costo marginal ≈ 0
- **LLM solo cuando importa** — disparador en eventos, no en cada step
- **Quantization** — **4-bit** / **8-bit** en GPU consumer

*Bien aplicadas, reducen el costo **10×–100×** sin pérdida apreciable.*

5. Aplicaciones y casos

LLM-ABM en el mundo real

Salud pública

Más allá del SIR clásico

- Simulación de pandemias con agentes que **deciden** vacunarse, distanciarse, ignorar reglas
- Cada agente tiene **creencias, fuente de información, red social**
- Permite estudiar:
 - Impacto de **mensajes** específicos de salud pública
 - **Polarización** alrededor de la vacunación
 - Diferencias entre poblaciones culturalmente distintas

Caso real: simulaciones COVID con agentes LLM en EE.UU. y UK (2024-2025).

Economía

Mercados con personalidades

- Traders con **personas** (greedy, contrarian, herding, value)
- Aparición espontánea de:
 - **Burbujas** y **crashes** sin shocks externos
 - **Colusión** implícita entre agentes que conversan
 - Asimetrías de información explotadas en lenguaje natural
- Permite testear **intervenciones regulatorias** (ej. **circuit breakers**)

Ciencias sociales

Opiniones, normas y polarización

- Modelos clásicos (DeGroot, Hegselmann-Krause) **predicen** opiniones como números
- Con LLMs: **opiniones como texto**, argumentos, contagio narrativo
- Casos típicos:
 - Aparición y muerte de **normas sociales**
 - **Cámaras de eco** en redes simuladas
 - Efecto de **influencers** vs **grassroots**
 - Cómo un **meme** se propaga y muta

Transporte y ciudades

Decisiones de movilidad con contexto

- Cada **commuter** es un agente con preferencias, hábitos y rutina
- Decisiones contextualizadas: clima, paro de subte, evento masivo, noticia
- Aplicaciones:
 - Diseño de **políticas de tráfico** (peajes dinámicos, carriles bici)
 - Evaluación de **shocks**: lluvia, accidentes, conciertos
 - Simulación de ciudades enteras con millones de agentes

Otros dominios prometedores

Donde LLM-ABM ya empezó a aparecer

- **Amenazas internas** — empleados con motivaciones que filtran información
- **Política** — votantes con creencias evolutivas, campañas adaptativas
- **Ecología** — cazadores furtivos, **rangers**, comunidades locales como agentes con discurso
- **Defensa / wargaming** — actores con doctrina propia y comunicación
- **Educación** — estudiantes simulados para evaluar curriculum
- **HCI / UX** — usuarios sintéticos para testear interfaces

Videos y referencias visuales

Lo que vamos a mirar

- **Smallville** (Park et al., 2023) — 25 agentes generativos viviendo
- **Mesa-LLM economic simulation** — mercado con 100 agentes LLM
- **Million-agent paper** (2025) — escalado masivo con archetypes
- **Concordia** (DeepMind) — **world simulation** con agentes situados

6. Desafíos y evaluación

¿Qué puede malir sal?

Costo computacional y escalabilidad

El elefante en la sala

- Una **step** de un agente generativo serio: **varias** llamadas al LLM
- $1000 \text{ agentes} \times 1000 \text{ steps} \times 5 \text{ prompts} = 5 \text{ M llamadas} \approx \text{mucho dinero}$
- Soluciones parciales:
 - Modelos locales (Ollama, vLLM)
 - Archetypes y clustering
 - LLM solo en eventos disparadores

Hoy, **el dominio de aplicación** define la estrategia: no hay solución universal.

Hallucinations, sesgos y reproducibilidad

Tres problemas que se mezclan

- **Alucinaciones** — el agente inventa hechos, herramientas, memorias
- **Sesgos** del modelo base — el LLM refleja la cultura de su corpus
- **No determinismo** — misma simulación, dos corridas, **resultados distintos**

Mitigaciones

- `temperature=0` cuando alcanza, sampling controlado cuando no
- Semillas explícitas, **function calling** con esquema validado
- Estadísticas sobre **decenas** de corridas, no una sola

Validación contra datos reales

¿La simulación reproduce el mundo?

- Comparar **distribuciones agregadas** simuladas vs observadas
 - tasas de adopción, **spread** de precios, polarización
- Comparar **trayectorias individuales** cuando hay datos:
 - movilidad anonimizada, encuestas longitudinales
- Tres tipos de validez:
 - **Behavioral realism** — los agentes actúan como humanos
 - **Outcome validity** — los resultados macro encajan
 - **Process validity** — los mecanismos son plausibles

Consideraciones éticas

No es solo un problema técnico

- **Privacidad** — ¿podemos entrenar agentes con perfiles reales?
- **Manipulación** — simulaciones que se usan para diseñar campañas dañinas
- **Antropomorfización** — confundir agentes simulados con personas
- **Bias amplification** — el modelo base reproduce y **acentúa** estereotipos
- **Dual-use** — un buen LLM-ABM social también es un buen **playbook** para influencia
- **Consentimiento** — ¿simular a personas vivas? ¿muertas?

*Toda simulación social con LLMs **debería** pasar por un comité de ética.*

Métricas de evaluación

¿Cómo medimos que funcionó?

Métrica	Qué mide
Alignment score	Coherencia del agente con su persona
Behavioral realism	Acciones plausibles vs estudio humano de control
Emergent validity	Aparecen patrones macro conocidos (Pareto, Zipf, etc.)
Memory consistency	El agente recuerda y no se contradice
Cost per insight	\$ por hallazgo útil

7. Actividad práctica

Hands-on guiado



INGENIERÍA

Elige tu propia aventura

Dos niveles de dificultad

Opción A · Fácil

- Modificar un agente simple en una **plantilla de Colab** que ya provee:
 - persona, memoria, prompt base
 - bucle de simulación
 - visualización
- Cambiá la **persona**, ajustá el prompt, observá la diferencia.

Opción B · Avanzada

- Diseñar un agente LLM **desde cero** para uno de:
 - **Tráfico urbano** — **commuters** con preferencias y rutina
 - **Negociación de mercado** — compradores y vendedores conversando
 - **Respuesta a desastre natural** — ciudadanos, **responders**, autoridades

Los 35 minutos

Pasos sugeridos

1. Definir persona y memoria inicial — 5 min
2. Escribir el **prompt template** — 10 min
3. Ejecutar mini-simulación — 10 min
4. Compartir resultados con la sala — 10 min

Trabajen **de a pares** o solos, como prefieran.
El que termine antes, juega con el **temperature** y compara.

Recursos

Para arrancar ahora

- **Plantilla Mesa-LLM** — `mesa_llm_template/` en el repo de la clase
- **Notebook Colab** — `mesa_llm_template/notebook_colab.ipynb` (subilo a colab.research.google.com)
- **Ejemplos runnables** — `examples/opcion_a_consumidor.py` y `opcion_b_trafico.py`
- **Prompts editables** — `prompts/consumer.txt`, `commuter.txt`, `trader.txt`, `disaster_citizen.txt`
- **Dataset de ejemplo** — `data/brand_opinions.csv` (32 opiniones sintéticas sobre 4 marcas)
- **API keys** — repartidas al iniciar; alternativa: `--provider mock` corre 100% offline

Tres modos de correr

Probá el que más te sirva hoy

```
# 1. Mock determinístico (sin internet, sin API key)
python examples/opcion_a_consumidor.py --provider mock --steps 15

# 2. Con OpenAI
$env:OPENAI_API_KEY = "sk-..."
python examples/opcion_a_consumidor.py --provider openai --model gpt-4o-mini

# 3. Con Ollama local (si tenés llama3.2 corriendo)
$env:OLLAMA_HOST = "http://localhost:11434"
python examples/opcion_a_consumidor.py --provider ollama
```

El `--provider mock` es **el plan B oficial**: si la wifi se cae o se rompe la API, la clase sigue.

8. Futuro y conclusión

Hacia dónde va esto

Tendencias 2026 – 2028

Lo que ya empieza a verse

- **Agentes multimodales** — visión + lenguaje + audio en el mismo loop
- **LLM + Reinforcement Learning** — agentes que **aprenden** dentro de la simulación
- **Simulaciones a escala de ciudades enteras** — millones de agentes con archetypes inteligentes
- **Benchmarks estandarizados** — **AgentBench**, **SimBench**, **SocialSim**
- **World models** — entornos simulados generados por LLMs
- **Reglamentación** — primeros marcos legales sobre simulaciones sociales

Recursos para seguir

Lecturas y repositorios

Papers fundacionales

- Park et al. (2023) — [Generative Agents: Interactive Simulacra of Human Behavior](#)
- Surveys de LLM-ABM 2024-2025 (Bonabeau, Wooldridge, Sun et al.)
- Concordia / DeepMind — [Generative Agent-Based Modeling](#)

Repos recomendados

- `joonspk-research/generative_agents`
- `microsoft/autogen`
- `projectmesa/mesa-llm`
- `langchain-ai/langgraph`

Llamado a la acción

Lo que querés hacer esta semana

- Cloná un repo de la lista
- Modificá una persona y corré 50 **steps**
- Compartí el resultado en el canal de la materia
- Si te entusiasmás: armá tu propio prompt para un dominio que te importe
- Mandá **issues** / **PRs** a Mesa-LLM — la comunidad es chica y receptiva

*El campo está **inmaduro** en el mejor sentido: hay mucho por hacer*

Estamos pasando de **simular comportamientos...**
a simular **mentes.**

¡Gracias!

¿Preguntas?



INGENIERÍA
